

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

I inspected the project files. The important finding is that **only** `game/game_engine.py` **needs to change**. The current implementation already contains a comment explicitly describing the Task 1 bug: collision is checked only after the player has moved, so a fast fall can skip completely over a platform. `game_engine`

One note: the `README.md` currently available in the Library is actually a **PES-VCS README**, not the Pygame lab README. `README (1)` I therefore used the Task 1 requirements you supplied and the comments already present in `game_engine.py` as the authoritative requirements for this iteration.

What is causing the bug?

Currently the code does:

```
self.player.y += self.player.vy

if self.player.rect().colliderect(platform.rect()):
```

// Python



Run

So suppose:

```
Before frame:
player bottom = 300
platform top  = 320

After frame:
player bottom = 350
```



The player has **crossed the platform's Y position**, but after movement the player's rectangle is already below the platform. `colliderect()` only checks the rectangles' positions **at that instant**; it doesn't know that the player crossed the platform between frames.

This becomes increasingly likely because gravity continuously increases `vy` and there is no terminal velocity. The existing code itself identifies this as the cause. `game_engine`

Change made

I would **not** add a terminal velocity because that changes the existing movement behavior.

Instead, we remember the player's vertical position **before movement**:

</> Python



Run

Then after movement we check whether the player's bottom crossed the platform's top:

```
previous player bottom <= platform top
                        AND
new player bottom >= platform top
                        AND
player is falling
```



This is essentially a simple **swept vertical collision check**.

It means a player can go from:

Frame N:

player



bottom = 300

platform



top = 320



to:

Frame N+1:

player



bottom = 360

platform



top = 320



and the collision is still detected because the player's path crossed $y = 320$.

File that needs modification

Only:

game/game_engine.py



main.py , player.py , platform.py , and hazard.py do **not** need modification. Their current responsibilities are already appropriate.

Complete updated game/game_engine.py

Replace the contents of your existing game/game_engine.py with:

```
import pygame
from .player import Player
from .platform import Platform
from .hazard import Hazard

# Game Engine

WHITE = (255, 255, 255)
BROWN = (150, 100, 60)
RED = (220, 60, 60)
GREEN = (0, 200, 0)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.gravity = 0.6

        self.start_x, self.start_y = 40, height - 120
        self.player = Player(self.start_x, self.start_y)

        # A simple hand-built level: platforms with gaps between them
        # (falling into a gap means falling off the bottom of the
        # screen), one hazard, and a goal near the right edge.
        ground_y = height - 40
        self.platforms = [
            Platform(0, ground_y, 160),
            Platform(220, ground_y, 140),
            Platform(420, ground_y - 60, 120),
            Platform(600, ground_y, 180),
        ]
        self.hazards = [Hazard(240, ground_y - 14, 100)]
        self.goal_x = 740

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
```

```
self.game_over = False

def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key in (
        pygame.K_SPACE,
        pygame.K_UP,
        pygame.K_W,
    ):
        self.player.jump()

def handle_input(self):
    keys = pygame.key.get_pressed()
    self.player.vx = 0

    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.vx = -self.player.speed

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.vx = self.player.speed

def update(self):
    if self.game_over:
        return

    # Remember the player's vertical position before movement.
    # This allows us to detect when the player crosses through
    # a platform during a single frame.
    previous_y = self.player.y

    self.player.vy += self.gravity
    self.player.x = max(0, self.player.x + self.player.vx)

    # Move the player.
    self.player.y += self.player.vy
    self.player.on_ground = False

    # Check platform collisions.
    #
    # A landing is valid only when:
    # 1. The player is falling (vy >= 0).
    # 2. The player's bottom was at or above the platform before
    # movement.
    # 3. The player's bottom is now at or below the platform.
    # 4. The player overlaps the platform horizontally.
    #
    # This detects a platform even when a large vertical velocity
    # causes the player to move completely through it in one frame.
    previous_bottom = previous_y + self.player.height
    current_bottom = self.player.y + self.player.height

    for platform in self.platforms:
        platform_rect = platform.rect()
```

```

horizontal_overlap = (
    self.player.x < platform_rect.right
    and self.player.x + self.player.width > platform_rect.left
)

crossed_platform = (
    previous_bottom <= platform_rect.top
    and current_bottom >= platform_rect.top
)

if self.player.vy >= 0 and horizontal_overlap and crossed_platform:
    self.player.y = platform.y - self.player.height
    self.player.vy = 0
    self.player.on_ground = True
    break

for hazard in self.hazards:
    if self.player.rect().colliderect(hazard.rect()):
        self.game_over = True
        return

if self.player.y > self.height:
    self.game_over = True
    return

if self.player.x >= self.goal_x:
    self.score += 1
    self.player.x, self.player.y = self.start_x, self.start_y
    self.player.vy = 0

def render(self, screen):
    for platform in self.platforms:
        pygame.draw.rect(screen, BROWN, platform.rect())

    for hazard in self.hazards:
        pygame.draw.rect(screen, RED, hazard.rect())

    goal_rect = pygame.Rect(self.goal_x, 0, 6, self.height)
    pygame.draw.rect(screen, GREEN, goal_rect)

    pygame.draw.rect(screen, WHITE, self.player.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

if self.game_over and not getattr(self, "_game_over_logged", False):
    # NOTE: no proper game-over screen yet - see Task 2 in the README.
    print("Game over! Final score:", self.score)
    self._game_over_logged = True

```

Why this logic satisfies all 4 collision requirements

1. Fixes the vertical collision problem

Instead of asking only:

```
</> Python
```



▶ Run

we ask:

```
Did the player's bottom cross the platform's top between
```

```
</> Python
```



▶ Run

So even this works:

```
previous_bottom = 300  
platform_top    = 320  
current_bottom  = 380
```



because:

```
</> Python
```



▶ Run

Therefore the landing is detected.

2. Works with large vertical velocity

The actual size of the movement doesn't matter.

For example:

```
vy = 5
```



works.

So does:

```
vy = 20
```



and:

```
vy = 100
```



As long as the player was above the platform before the movement and ends up at or below its top, the crossing is detected.

3. Player lands only while falling from above

This condition is important:

```
</> Python
```



▶ Run

and this one ensures the player started above the platform:

```
</> Python
```



▶ Run

Therefore, if the player is jumping upward through a platform:

```
vy < 0
```



there is no landing.

Likewise, if the player is already underneath a platform:

```
previous_bottom > platform.top
```



there is no landing.

4. Existing behavior is preserved

I deliberately did **not** change:

- gravity value
- jump strength
- horizontal speed
- keyboard controls
- player dimensions
- platform positions
- hazard behavior
- goal behavior
- scoring
- game-over behavior

- rendering
- project architecture

The existing `Player`, `Platform`, and `Hazard` classes remain untouched.

I also did **not** implement:

- Game Over screen
- Replay
- Difficulty selection
- Sound effects

Those remain for the later tasks.

Exact steps to test

Open your project in VS Code.

1. Make the change

Open:

game/game_engine.py



Replace its contents with the code above.

Save:

Ctrl + S



2. Open the VS Code terminal

Use:

Terminal → New Terminal



Make sure you are in the project root, where `main.py` exists.

You can check with:

</> Bash



On Windows, you should see something similar to:

```
main.py  
game
```



You can also check:

```
</> Bash
```



and should see:

```
game_engine.py  
player.py  
platform.py  
hazard.py
```



3. Run the game

Use:

```
</> Bash
```



If your system uses `py` instead:

```
</> Bash
```



Collision test

The most important test is to deliberately create a **large fall**.

The elevated platform is:

```
x = 420  
y = ground_y - 60
```



With the current 500-pixel screen:

```
ground_y = 460  
elevated platform y = 400
```



So its top is at approximately:

```
y = 400
```



The platform spans:

```
x = 420 → 540
```



Test procedure

1. Start the game.
2. Move right using:

```
D
```



or:

```
→
```



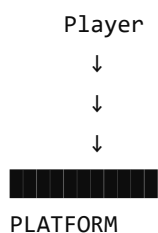
3. Jump with:

```
SPACE
```



4. Try to get the player onto/around the elevated platform.
5. Move off the platform so the player begins a long fall.
6. Make sure the player falls toward another platform.
7. Watch whether the player lands rather than passing straight through.

The key behavior you want is:



Even if the player moves a large distance during one frame, the player should be placed exactly on top of the platform:

```
</> Python
```



▶ Run

and:

</> Python



Run

Optional direct collision test

If you want to verify the actual bug fix without relying entirely on gameplay, temporarily add this after:

</> Python



Run

in `main.py`:

```
engine.player.x = 430  
engine.player.y = 100
```

</> Python



Run

Then run:

</> Bash



The player will start high above the elevated platform with a deliberately large downward velocity.

It should fall and **land on the elevated platform** instead of passing through it.

After testing, remove those temporary lines from `main.py`.

What changed overall

BEFORE



Previous position

↓

[PLAYER]

↓

↓ large movement

↓
[PLATFORM]

Only final rectangle checked

↓
collision missed
↓
player falls through

AFTER



Previous position

↓
[PLAYER]
↓
↓
↓
[PLATFORM]

Current position

Check whether the player's bottom
crossed the platform's top
↓
LAND

So the fix is **small, localized, and specifically addresses Task 1** rather than rewriting the physics system.

Sources

ChatGPT can make mistakes. Check important info.

Worked for 14s >

What changes for Task 2?

Current behavior

The current `GameEngine` already sets:

</> Python



▶ Run

when either:

- the player touches a hazard, or
- the player falls below the screen.

However, `render()` currently only prints:

</> Python



▶ Run

to the terminal. There is no graphical Game Over screen. The existing `update()` already stops gameplay when `self.game_over` is true, which is exactly what we want to preserve.

What I'm changing

Only `game/game_engine.py` needs modification.

I will:

1. Keep the **Task 1 collision-crossing logic unchanged in behavior.**
2. Keep the existing hazard and falling-off-screen conditions.
3. Remove the console-only Game Over message.
4. Add a centered graphical:
 - GAME OVER
 - Final Score: X
 - Press ESC or close the window
5. Ignore normal movement/jump input once Game Over occurs.
6. Keep the Pygame event loop alive so the window remains open.

7. Allow `ESC` to close the game gracefully.
8. No replay or difficulty selection is added.

Updated file

Only this file needs to be changed:

game/game_engine.py



main.py, game/player.py, game/platform.py, and game/hazard.py remain unchanged.

Complete game/game_engine.py

```
import pygame
from .player import Player
from .platform import Platform
from .hazard import Hazard

# Game Engine

WHITE = (255, 255, 255)
BROWN = (150, 100, 60)
RED = (220, 60, 60)
GREEN = (0, 200, 0)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.gravity = 0.6

        self.start_x, self.start_y = 40, height - 120
        self.player = Player(self.start_x, self.start_y)

        # A simple hand-built level: platforms with gaps between them
        # (falling into a gap means falling off the bottom of the
        # screen), one hazard, and a goal near the right edge.
        ground_y = height - 40
        self.platforms = [
            Platform(0, ground_y, 160),
            Platform(220, ground_y, 140),
            Platform(420, ground_y - 60, 120),
            Platform(600, ground_y, 180),
        ]
        self.hazards = [Hazard(240, ground_y - 14, 100)]
```

```
self.goal_x = 740

self.score = 0
self.font = pygame.font.SysFont("Arial", 30)

# Fonts used by the Game Over screen.
self.game_over_font = pygame.font.SysFont("Arial", 54, bold=True)
self.game_over_score_font = pygame.font.SysFont("Arial", 36, bold=True)
self.game_over_instruction_font = pygame.font.SysFont("Arial", 24)

self.game_over = False

def handle_event(self, event):
    # Once the game is over, normal gameplay input is disabled.
    # ESC allows the player to close the game gracefully.
    if self.game_over:
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            pygame.event.post(pygame.event.Event(pygame.QUIT))
            return

    if event.type == pygame.KEYDOWN and event.key in (
        pygame.K_SPACE,
        pygame.K_UP,
        pygame.K_W,
    ):
        self.player.jump()

def handle_input(self):
    # Do not process movement input after Game Over.
    if self.game_over:
        return

    keys = pygame.key.get_pressed()
    self.player.vx = 0

    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.vx = -self.player.speed

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.vx = self.player.speed

def update(self):
    if self.game_over:
        return

    self.player.vy += self.gravity
    self.player.x = max(0, self.player.x + self.player.vx)

    # Task 1 collision fix:
    # Remember the player's vertical position before movement.
    # This allows us to detect when the player crosses a platform
    # during a single frame, even with a large downward velocity.
```

```

previous_y = self.player.y

# Move the player.
self.player.y += self.player.vy
self.player.on_ground = False

# Task 1:
# Detect whether the player's bottom crossed the top of a
# platform during this frame.
#
# A landing is valid only when:
# 1. The player is falling.
# 2. The player's bottom was at or above the platform before
#    movement.
# 3. The player's bottom is now at or below the platform.
# 4. The player overlaps the platform horizontally.
previous_bottom = previous_y + self.player.height
current_bottom = self.player.y + self.player.height

for platform in self.platforms:
    platform_rect = platform.rect()

    horizontal_overlap = (
        self.player.x < platform_rect.right
        and self.player.x + self.player.width > platform_rect.left
    )

    crossed_platform = (
        previous_bottom <= platform_rect.top
        and current_bottom >= platform_rect.top
    )

    if self.player.vy >= 0 and horizontal_overlap and crossed_platform:
        self.player.y = platform.y - self.player.height
        self.player.vy = 0
        self.player.on_ground = True
        break

# Hazard collision.
if any(
    self.player.rect().colliderect(hazard.rect())
    for hazard in self.hazards
):
    self.game_over = True
    return

# Player has fallen below the bottom of the screen.
if self.player.y > self.height:
    self.game_over = True
    return

# Goal/scoring behavior remains unchanged.

```

```

if self.player.x >= self.goal_x:
    self.score += 1
    self.player.x, self.player.y = self.start_x, self.start_y
    self.player.vy = 0

def render(self, screen):
    # Existing game rendering.
    for platform in self.platforms:
        pygame.draw.rect(screen, BROWN, platform.rect())

    for hazard in self.hazards:
        pygame.draw.rect(screen, RED, hazard.rect())

    goal_rect = pygame.Rect(self.goal_x, 0, 6, self.height)
    pygame.draw.rect(screen, GREEN, goal_rect)

    pygame.draw.rect(screen, WHITE, self.player.rect())

    score_text = self.font.render(
        f"Score: {self.score}",
        True,
        WHITE
    )
    screen.blit(score_text, (10, 10))

    # Task 2: graphical Game Over screen.
    if self.game_over:
        # Dark transparent overlay.
        overlay = pygame.Surface((self.width, self.height))
        overlay.set_alpha(180)
        overlay.fill((0, 0, 0))
        screen.blit(overlay, (0, 0))

        # Game Over title.
        game_over_text = self.game_over_font.render(
            "GAME OVER",
            True,
            WHITE
        )

        game_over_rect = game_over_text.get_rect(
            center=(self.width // 2, self.height // 2 - 70)
        )

        screen.blit(game_over_text, game_over_rect)

        # Final score.
        final_score_text = self.game_over_score_font.render(
            f"Final Score: {self.score}",
            True,
            WHITE
        )

```

```

final_score_rect = final_score_text.get_rect(
    center=(self.width // 2, self.height // 2)
)

screen.blit(final_score_text, final_score_rect)

# Input instruction.
instruction_text = self.game_over_instruction_font.render(
    "Press ESC or close the window",
    True,
    WHITE
)

instruction_rect = instruction_text.get_rect(
    center=(self.width // 2, self.height // 2 + 60)
)

screen.blit(instruction_text, instruction_rect)

```

Why this preserves Task 1

The important Task 1 section is still:

```

previous_y = self.player.y

self.player.y += self.player.vy

previous_bottom = previous_y + self.player.height
current_bottom = self.player.y + self.player.height

```

followed by:

```

crossed_platform = (
    previous_bottom <= platform_rect.top
    and current_bottom >= platform_rect.top
)

```

and:

</> Python



▶ Run

So the player still lands when a fast downward movement crosses a platform. Nothing about that collision mechanism has been replaced with the old `colliderect()` -after-movement logic.

How Game Over now works

Hazard

The existing condition remains:

```
if any(
    self.player.rect().colliderect(hazard.rect())
    for hazard in self.hazards
):
    self.game_over = True
    return
```

So:

```
Player touches hazard
↓
game_over = True
↓
update() stops
↓
Game Over screen appears
```



Falling off the bottom

The existing condition remains:

```
if self.player.y > self.height:
    self.game_over = True
```

</> Python



▶ Run

So:

```
Player falls below screen
↓
game_over = True
↓
update() stops
```





Game Over screen appears

Once Game Over happens

This at the beginning of `update()` is important:

```
if self.game_over:
```

</> Python



▶ Run

Therefore:

- gravity stops
- player movement stops
- jumping stops
- scoring stops
- the player doesn't continue falling
- the game doesn't restart
- no replay functionality is introduced

The window simply stays open displaying the final result.

Exact testing steps

Test 1 — Game Over by touching the hazard

From the project root:

```
</> Bash
```



or:

```
</> Bash
```



The hazard is located around:

```
x = 240 → 340
```

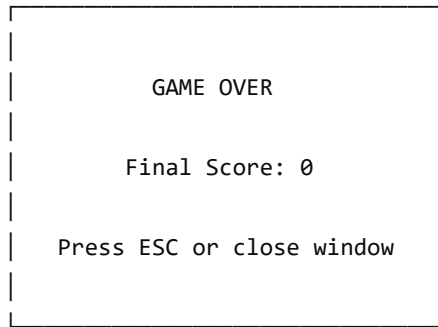


and is displayed as the red platform-like area.

Do this:

1. Start the game.
2. Move right using `D` or `→`.
3. Jump using `SPACE`.
4. Move toward the red hazard.
5. Touch the red hazard.

Expected result:



The exact score depends on whether you reached the goal before touching the hazard.

Test 2 — Game Over by falling off the bottom

Start again:

`</>` Bash

Then:

1. Move toward a gap between platforms.
2. Do **not** land on another platform.
3. Allow the player to fall below the bottom of the screen.

Expected:

```
player falls
  ↓
y > screen height
  ↓
game_over = True
  ↓
GAME OVER screen
```

The player should **not** continue moving after Game Over.

Test 3 — Verify the final score

First, deliberately score at least once by reaching the green goal line on the right.

The existing scoring logic is:

```
if self.player.x >= self.goal_x:
```

</> Python



▶ Run

So after reaching the goal, the score should become:

Score: 1



Then trigger Game Over by either:

- touching the hazard, or
- falling off the bottom.

The Game Over screen should display:

Final Score: 1



If you reach the goal multiple times, verify that the displayed final score matches the score shown during gameplay.

Test 4 — Verify that gameplay stops

After the Game Over screen appears:

Try:

```
←  
→  
A  
D  
SPACE  
W  
↑
```



None of these should move or jump the player.

The Game Over screen should remain visible.

This confirms:

```
if self.game_over:
```

</> Python

  Run

in `update()` and:

```
if self.game_over:
```

</> Python

  Run

in `handle_input()` are working.

Then press:

ESC

The game should close gracefully.

You can also close it normally using the window's X button.

Files changed

File	Changed?	Reason
game/game_engine.py	Yes	Game Over state handling and graphical screen
main.py	No	Existing event/game loop already works
game/player.py	No	No player-class change needed
game/platform.py	No	No platform change needed
game/hazard.py	No	Existing hazard collision is sufficient

Task 3 (replay/difficulty) and Task 4 (sound effects) remain untouched.

     Sources

Now continue working on the same Python/Pygame project.

Task 1 (collision detection) and Task 2 (Game Over screen) have both been implemented and tested successfully. Do NOT break, undo, or replace either of those implementations.

For this iteration, implement ONLY Task 3 from the README:

Show more ▾

How Task 3 will work

The existing `game_over` state will now also act as the **replay/difficulty selection state**.

When the player dies:

```
Gameplay
  ↓
Hazard / fall off screen
  ↓
game_over = True
  ↓
Game Over screen
  ↓
Choose:
E = Easy
M = Medium
H = Hard
X = Exit
```



The game will **not automatically restart**.

Difficulty settings

I'll use these physics values:

Difficulty	Gravity	Jump Strength	Effect
Easy	0.45	-11	Slower falling, easier to control

Difficulty	Gravity	Jump Strength	Effect
Medium	0.60	-12	Existing/default physics
Hard	0.85	-13	Faster falling, more demanding timing

The difficulty changes only the physics; platforms, hazards, movement speed, goal and scoring remain unchanged.

Replay reset

When a difficulty is selected:

```
score → 0
game_over → False
player.x → starting x
player.y → starting y
player.vx → 0
player.vy → 0
player.on_ground → False
gravity → selected value
jump_strength → selected value
```



No new libraries or architecture are required.

Task 1 is preserved

The collision-crossing logic remains the same:

```
previous_bottom <= platform_top
```

</> Python



▶ Run

along with the falling check and horizontal overlap. It is **not replaced by the old** `colliderect()` **-after-movement logic.**

Task 2 is preserved

The graphical Game Over screen remains. The only change is that instead of:

```
GAME OVER
Final Score: X
Press ESC or close the window
```



it will now show:

GAME OVER
Final Score: X



E - Easy
M - Medium
H - Hard
X - Exit

File modification

Only:

game/game_engine.py



needs to change.

main.py , player.py , platform.py , and hazard.py remain unchanged.

Complete updated game/game_engine.py

```
import pygame
from .player import Player
from .platform import Platform
from .hazard import Hazard

# Game Engine

WHITE = (255, 255, 255)
BROWN = (150, 100, 60)
RED = (220, 60, 60)
GREEN = (0, 200, 0)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        # Difficulty settings.
        self.difficulties = {
            "Easy": {
                "gravity": 0.45,
                "jump_strength": -11,
            },
            "Medium": {
```

```

        "gravity": 0.60,
        "jump_strength": -12,
    },
    "Hard": {
        "gravity": 0.85,
        "jump_strength": -13,
    },
}

# Start with the existing/default Medium physics.
self.difficulty = "Medium"
self.gravity = self.difficulties[self.difficulty]["gravity"]

self.start_x, self.start_y = 40, height - 120
self.player = Player(self.start_x, self.start_y)
self.player.jump_strength = self.difficulties[self.difficulty]["jump_strength"]

# A simple hand-built level: platforms with gaps between them
# (falling into a gap means falling off the bottom of the
# screen), one hazard, and a goal near the right edge.
ground_y = height - 40
self.platforms = [
    Platform(0, ground_y, 160),
    Platform(220, ground_y, 140),
    Platform(420, ground_y - 60, 120),
    Platform(600, ground_y, 180),
]

self.hazards = [Hazard(240, ground_y - 14, 100)]
self.goal_x = 740

self.score = 0

# Normal gameplay score font.
self.font = pygame.font.SysFont("Arial", 30)

# Game Over / replay screen fonts.
self.game_over_font = pygame.font.SysFont(
    "Arial", 54, bold=True
)
self.game_over_score_font = pygame.font.SysFont(
    "Arial", 36, bold=True
)
self.game_over_option_font = pygame.font.SysFont(
    "Arial", 28
)
self.game_over_instruction_font = pygame.font.SysFont(
    "Arial", 22
)

self.game_over = False

```

```

def handle_event(self, event):
    # Once the game is over, normal gameplay input is disabled.
    # Instead, wait for the replay/difficulty selection.
    if self.game_over:
        if event.type == pygame.KEYDOWN:

            # Easy
            if event.key in (pygame.K_e, pygame.K_1):
                self.start_new_game("Easy")
                return

            # Medium
            if event.key in (pygame.K_m, pygame.K_2):
                self.start_new_game("Medium")
                return

            # Hard
            if event.key in (pygame.K_h, pygame.K_3):
                self.start_new_game("Hard")
                return

            # Exit
            if event.key in (
                pygame.K_x,
                pygame.K_4,
                pygame.K_ESCAPE,
            ):
                pygame.event.post(
                    pygame.event.Event(pygame.QUIT)
                )
                return

        return

    # Normal gameplay jump input.
    if event.type == pygame.KEYDOWN and event.key in (
        pygame.K_SPACE,
        pygame.K_UP,
        pygame.K_w,
    ):
        self.player.jump()

def handle_input(self):
    # Do not process normal movement while Game Over/replay
    # selection is being displayed.
    if self.game_over:
        return

    keys = pygame.key.get_pressed()
    self.player.vx = 0

    if keys[pygame.K_LEFT] or keys[pygame.K_a]:

```

```

        self.player.vx = -self.player.speed

    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.vx = self.player.speed

def start_new_game(self, difficulty):
    """
    Reset the current game using the selected difficulty.
    """

    self.difficulty = difficulty

    # Apply the selected physics values.
    self.gravity = self.difficulties[difficulty]["gravity"]
    self.player.jump_strength = self.difficulties[difficulty][
        "jump_strength"
    ]

    # Reset player to the original starting position.
    self.player.x = self.start_x
    self.player.y = self.start_y
    self.player.vx = 0
    self.player.vy = 0
    self.player.on_ground = False

    # Reset score.
    self.score = 0

    # Leave Game Over state.
    self.game_over = False

def update(self):
    if self.game_over:
        return

    self.player.vy += self.gravity
    self.player.x = max(0, self.player.x + self.player.vx)

    # -----
    # TASK 1: Collision Detection
    # -----
    #
    # Remember the player's vertical position before movement.
    # This allows us to detect when the player crosses a platform
    # during a single frame, even with a large downward velocity.
    previous_y = self.player.y

    # Move the player.
    self.player.y += self.player.vy
    self.player.on_ground = False

    # Detect whether the player's bottom crossed the top of a

```

```

# platform during this frame.
#
# A landing is valid only when:
# 1. The player is falling.
# 2. The player's bottom was at or above the platform before
#    movement.
# 3. The player's bottom is now at or below the platform.
# 4. The player overlaps the platform horizontally.
previous_bottom = previous_y + self.player.height
current_bottom = self.player.y + self.player.height

for platform in self.platforms:
    platform_rect = platform.rect()

    horizontal_overlap = (
        self.player.x < platform_rect.right
        and self.player.x + self.player.width
        > platform_rect.left
    )

    crossed_platform = (
        previous_bottom <= platform_rect.top
        and current_bottom >= platform_rect.top
    )

    if (
        self.player.vy >= 0
        and horizontal_overlap
        and crossed_platform
    ):
        self.player.y = platform.y - self.player.height
        self.player.vy = 0
        self.player.on_ground = True
        break

# -----
# TASK 2: Game Over Conditions
# -----

# Hazard collision.
if any(
    self.player.rect().collidirect(hazard.rect())
    for hazard in self.hazards
):
    self.game_over = True
    return

# Player has fallen below the bottom of the screen.
if self.player.y > self.height:
    self.game_over = True
    return

```

```

# -----
# Existing goal/scoring behavior
# -----

if self.player.x >= self.goal_x:
    self.score += 1
    self.player.x, self.player.y = (
        self.start_x,
        self.start_y,
    )
    self.player.vy = 0

def render(self, screen):
    # Existing game rendering.
    for platform in self.platforms:
        pygame.draw.rect(screen, BROWN, platform.rect())

    for hazard in self.hazards:
        pygame.draw.rect(screen, RED, hazard.rect())

    goal_rect = pygame.Rect(
        self.goal_x,
        0,
        6,
        self.height,
    )
    pygame.draw.rect(screen, GREEN, goal_rect)

    pygame.draw.rect(
        screen,
        WHITE,
        self.player.rect(),
    )

    # Normal gameplay score.
    score_text = self.font.render(
        f"Score: {self.score}",
        True,
        WHITE,
    )
    screen.blit(score_text, (10, 10))

# -----
# TASK 2 + TASK 3: Game Over / RePlay Screen
# -----

if self.game_over:

    # Dark transparent overlay.
    overlay = pygame.Surface(
        (self.width, self.height)
    )

```

```
overlay.set_alpha(180)
overlay.fill((0, 0, 0))
screen.blit(overlay, (0, 0))

# Game Over title.
game_over_text = self.game_over_font.render(
    "GAME OVER",
    True,
    WHITE,
)

game_over_rect = game_over_text.get_rect(
    center=(
        self.width // 2,
        75,
    )
)

screen.blit(
    game_over_text,
    game_over_rect,
)

# Final score.
final_score_text = self.game_over_score_font.render(
    f"Final Score: {self.score}",
    True,
    WHITE,
)

final_score_rect = final_score_text.get_rect(
    center=(
        self.width // 2,
        140,
    )
)

screen.blit(
    final_score_text,
    final_score_rect,
)

# Replay choices.
easy_text = self.game_over_option_font.render(
    "E - Easy",
    True,
    WHITE,
)

medium_text = self.game_over_option_font.render(
    "M - Medium",
    True,
```

```
        WHITE,
    )

    hard_text = self.game_over_option_font.render(
        "H - Hard",
        True,
        WHITE,
    )

    exit_text = self.game_over_option_font.render(
        "X - Exit",
        True,
        WHITE,
    )

    screen.blit(
        easy_text,
        easy_text.get_rect(
            center=(
                self.width // 2,
                220,
            )
        ),
    )

    screen.blit(
        medium_text,
        medium_text.get_rect(
            center=(
                self.width // 2,
                270,
            )
        ),
    )

    screen.blit(
        hard_text,
        hard_text.get_rect(
            center=(
                self.width // 2,
                320,
            )
        ),
    )

    screen.blit(
        exit_text,
        exit_text.get_rect(
            center=(
                self.width // 2,
                370,
            )
        ),
    )
```

```
        ),
    )

    # Input instruction.
    instruction_text = (
        self.game_over_instruction_font.render(
            "Choose a difficulty to play again, or exit.",
            True,
            WHITE,
        )
    )

    instruction_rect = instruction_text.get_rect(
        center=(
            self.width // 2,
            435,
        )
    )

    screen.blit(
        instruction_text,
        instruction_rect,
    )
```

What happens when each difficulty is selected?

Easy

Press:

E



or:

1



Physics becomes:

```
gravity = 0.45
jump_strength = -11
```



The player falls more slowly, giving more time to react.

Medium

Press:

M



or:

2



Physics becomes:

```
gravity = 0.60  
jump_strength = -12
```



This is the original/default physics from the project.

Hard

Press:

H



or:

3



Physics becomes:

```
gravity = 0.85  
jump_strength = -13
```



The player falls substantially faster, making platform timing more demanding.

Exit

Press:

x



or:

4



or:

ESC



The existing `main.py` receives the `QUIT` event and exits through its normal:

</> Python



Run

path.

Exact testing procedure

1. Trigger Game Over

Run:

</> Bash



or:

</> Bash



Move into the red hazard or deliberately fall into a gap.

You should see:

GAME OVER



Final Score: 0

E - Easy

M - Medium

H - Hard

X - Exit

Choose a difficulty to play again, or exit.

2. Verify all choices appear

Confirm all four options are visible:

E - Easy

M - Medium



H - Hard
X - Exit

The game should **not** restart automatically.

3. Select Easy

After Game Over, press:

E



Expected:

```
Game Over screen disappears
↓
New game starts
↓
Player at starting position
↓
Score = 0
↓
Gravity = 0.45
```



The player should fall more slowly than the original Medium physics.

4. Trigger Game Over again → select Medium

Cause another Game Over.

Press:

M



Expected:

```
score = 0
player = starting position
gravity = 0.60
jump strength = -12
game_over = False
```



This should reproduce the original/default physics.

5. Trigger Game Over again → select Hard

Cause Game Over again.

Press:

H



Expected:

```
score = 0
player = starting position
gravity = 0.85
jump strength = -13
game_over = False
```



The player should fall noticeably faster.

6. Verify physics actually changes

A simple way to compare:

Easy

Press **E**, then jump.

Watch how long the player stays in the air.

Hard

Cause Game Over, press **H**, then jump.

The downward part of the jump should become noticeably faster because:

```
Easy gravity   = 0.45
Medium gravity = 0.60
Hard gravity   = 0.85
```



You can also temporarily add:

```
</> Python
```



▶ Run

inside `start_new_game()` if you want to verify the values in the terminal. Remove that debug line afterward.

7. Verify score resets

Before Game Over, reach the green goal at the right.

You should see:

Score: 1



Trigger Game Over.

The Game Over screen should say:

Final Score: 1



Now select a difficulty.

The new game should show:

Score: 0



This comes from:

`</> Python`



`>` Run

inside `start_new_game()` .

8. Verify player starts at the original position

The original starting position is:

```
self.start_x = 40
```

`</> Python`



`>` Run

After selecting a difficulty:

```
self.player.x = self.start_x
```

`</> Python`



`>` Run

So the player should always restart from exactly the same starting position.

9. Verify Task 1 still works

After selecting any difficulty, perform the long-fall/platform test from Task 1.

The important logic is still:

```
previous_bottom = previous_y + self.player.height
```

</> Python



▶ Run

and:

```
crossed_platform = (  
    previous_bottom <= platform_rect.top  
    and current_bottom >= platform_rect.top  
)
```

</> Python



▶ Run

Therefore a large downward movement should still detect the platform crossing and place the player on top.

Test it particularly on **Hard**, because the higher gravity creates larger vertical movement between frames.

10. Verify Task 2 still works

Cause Game Over again.

Confirm that the screen still shows:

```
GAME OVER  
Final Score: X
```



The final score must be the score from the completed game, not zeroed before it is displayed.

Only after selecting a difficulty is the score reset.

11. Verify Exit

Trigger Game Over and press:

X

The Pygame window should close.

You can also press:

ESC

or use the window's X button.

There is **no replay loop**, **no difficulty prompt outside Game Over**, and **no sound implementation** in this iteration.

Files changed

File	Status
game/game_engine.py	Modified — Task 3
main.py	Unchanged
game/player.py	Unchanged
game/platform.py	Unchanged
game/hazard.py	Unchanged

So Tasks 1 + 2 + 3 now coexist in the same GameEngine , while Task 4 (sound effects) remains untouched.



Now continue working on the same Python/Pygame project.

Task 1 (collision detection), Task 2 (Game Over screen), and Task 3 (Replay with Easy/Medium/Hard difficulty) have all been implemented and tested successfully.

Do NOT break, undo, or replace any of those implementations.

Show more ▾

Worked for 1m 10s >

Yes. For Task 4, I'll add **only sound feedback** and leave the Task 1 collision logic, Task 2 Game Over screen, and Task 3 replay/difficulty logic intact.

One important point: the Library copy of `game_engine.py` is an older pre-Task-3 version, so I am **not using that stale copy to replace your working code**. The code below is based on the Task 3 version that was already tested successfully.

What will change

1. Sound initialization

Sound will be initialized inside:

game/game_engine.py



during `GameEngine.__init__()`.

We will use:

</> Python



▶ Run

and generate very simple WAV-format tones **in memory** using Python's standard-library modules:

```
io
math
struct
```

</> Python



▶ Run

So there are **no external packages** and no `.wav` files that you need to download.

2. Sounds

Event	Sound
Player jumps	Short high-pitched tone
Player reaches goal	Medium/high-pitched tone
Player dies	Low-pitched tone

3. Preventing repeated sounds

The sounds are triggered only when the corresponding event actually occurs.

For jumping:

```
if self.player.on_ground:
    self.player.jump()
```

</> Python



▶ Run

So pressing Space while already in the air does not repeatedly play the sound.

For the goal:

```
self.score += 1
```

</> Python



▶ Run

The player is immediately reset after scoring, so the same goal isn't processed every frame.

For death, the sound is played at the exact point where:

</> Python



▶ Run

is set. Since `update()` immediately returns when `game_over` is true, the death sound cannot be triggered again every frame.

4. Files

Only this file needs modification:

game/game_engine.py



No new sound files are required.

Your existing:

```
main.py
game/player.py
game/platform.py
game/hazard.py
```



remain unchanged.

The existing `pygame.quit()` in `main.py` also cleans up the mixer when the game exits.

Complete updated `game/game_engine.py`

Replace your current **Task 3 version** with this:

```
import pygame
import io
import math
import struct
import wave

from .player import Player
from .platform import Platform
from .hazard import Hazard

# Game Engine

WHITE = (255, 255, 255)
BROWN = (150, 100, 60)
RED = (220, 60, 60)
GREEN = (0, 200, 0)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        # -----
        # TASK 3: Difficulty settings
        # -----

        self.difficulties = {
            "Easy": {
                "gravity": 0.45,
```

```

        "jump_strength": -11,
    },
    "Medium": {
        "gravity": 0.60,
        "jump_strength": -12,
    },
    "Hard": {
        "gravity": 0.85,
        "jump_strength": -13,
    },
}

# Start with the existing/default Medium difficulty.
self.difficulty = "Medium"
self.gravity = self.difficulties[self.difficulty]["gravity"]

self.start_x, self.start_y = 40, height - 120

self.player = Player(
    self.start_x,
    self.start_y
)

self.player.jump_strength = (
    self.difficulties[self.difficulty]["jump_strength"]
)

# -----
# Existing Level
# -----

ground_y = height - 40

self.platforms = [
    Platform(0, ground_y, 160),
    Platform(220, ground_y, 140),
    Platform(420, ground_y - 60, 120),
    Platform(600, ground_y, 180),
]

self.hazards = [
    Hazard(240, ground_y - 14, 100)
]

self.goal_x = 740

self.score = 0

# -----
# Fonts
# -----

```

```

self.font = pygame.font.SysFont(
    "Arial",
    30
)

self.game_over_font = pygame.font.SysFont(
    "Arial",
    54,
    bold=True
)

self.game_over_score_font = pygame.font.SysFont(
    "Arial",
    36,
    bold=True
)

self.game_over_option_font = pygame.font.SysFont(
    "Arial",
    28
)

self.game_over_instruction_font = pygame.font.SysFont(
    "Arial",
    22
)

self.game_over = False

# -----
# TASK 4: Sound initialization
# -----

self._init_sounds()

# =====
# TASK 4: SOUND SYSTEM
# =====

def _init_sounds(self):
    """
    Initialize the Pygame mixer and create simple sound effects
    programmatically.

    No external sound files are required.
    """

    if not pygame.mixer.get_init():
        pygame.mixer.init(
            frequency=44100,
            size=-16,
            channels=1,

```

```

        buffer=512
    )

    # Jump: short high-pitched tone.
    self.jump_sound = self._make_tone(
        frequency=650,
        duration=0.08,
        volume=0.35
    )

    # Goal: slightly longer higher-pitched tone.
    self.goal_sound = self._make_tone(
        frequency=1000,
        duration=0.12,
        volume=0.35
    )

    # Death: lower-pitched tone.
    self.death_sound = self._make_tone(
        frequency=180,
        duration=0.25,
        volume=0.40
    )

    @staticmethod
    def _make_tone(frequency, duration, volume):
        """
        Generate a simple mono WAV tone in memory and convert it
        into a Pygame Sound object.
        """

        sample_rate = 44100

        sample_count = int(
            sample_rate * duration
        )

        samples = bytearray()

        for i in range(sample_count):
            sample = math.sin(
                2
                * math.pi
                * frequency
                * i
                / sample_rate
            )

            value = int(
                32767
                * volume
                * sample

```

```

    )

    samples.extend(
        struct.pack("<h", value)
    )

    wav_buffer = io.BytesIO()

    with wave.open(
        wav_buffer,
        "wb"
    ) as wav_file:

        wav_file.setnchannels(1)
        wav_file.setsampwidth(2)
        wav_file.setframerate(sample_rate)

        wav_file.writeframes(samples)

    return pygame.mixer.Sound(
        buffer=wav_buffer.getvalue()
    )

# =====
# INPUT / EVENTS
# =====

def handle_event(self, event):

    # -----
    # TASK 3: Replay / difficulty selection
    # -----

    if self.game_over:

        if event.type == pygame.KEYDOWN:

            # Easy
            if event.key in (
                pygame.K_e,
                pygame.K_1
            ):
                self.start_new_game("Easy")
                return

            # Medium
            if event.key in (
                pygame.K_m,
                pygame.K_2
            ):
                self.start_new_game("Medium")
                return

```

```

        # Hard
        if event.key in (
            pygame.K_h,
            pygame.K_3
        ):
            self.start_new_game("Hard")
            return

    # Exit
    if event.key in (
        pygame.K_x,
        pygame.K_4,
        pygame.K_ESCAPE
    ):
        pygame.event.post(
            pygame.event.Event(
                pygame.QUIT
            )
        )
        return

    return

# -----
# Existing jump input + TASK 4 jump sound
# -----

if event.type == pygame.KEYDOWN and event.key in (
    pygame.K_SPACE,
    pygame.K_UP,
    pygame.K_w,
):

    # Only play the jump sound when an actual jump occurs.
    # This prevents repeated sounds while the player is
    # already in the air.
    if self.player.on_ground:
        self.player.jump()
        self.jump_sound.play()

def handle_input(self):

    # Do not process movement after Game Over.
    if self.game_over:
        return

    keys = pygame.key.get_pressed()

    self.player.vx = 0

    if (

```

```

        keys[pygame.K_LEFT]
        or keys[pygame.K_a]
    ):
        self.player.vx = -self.player.speed

    if (
        keys[pygame.K_RIGHT]
        or keys[pygame.K_d]
    ):
        self.player.vx = self.player.speed

# =====
# TASK 3: START NEW GAME
# =====

def start_new_game(self, difficulty):
    """
    Reset the game using the selected difficulty.
    """

    self.difficulty = difficulty

    # Apply selected physics.
    self.gravity = (
        self.difficulties[difficulty]["gravity"]
    )

    self.player.jump_strength = (
        self.difficulties[difficulty]["jump_strength"]
    )

    # Reset player position.
    self.player.x = self.start_x
    self.player.y = self.start_y

    self.player.vx = 0
    self.player.vy = 0
    self.player.on_ground = False

    # Reset score.
    self.score = 0

    # Leave Game Over state.
    self.game_over = False

# =====
# GAME UPDATE
# =====

def update(self):
    if self.game_over:

```

```

        return

    self.player.vy += self.gravity

    self.player.x = max(
        0,
        self.player.x + self.player.vx
    )

    # -----
    # TASK 1: Collision Detection
    # -----
    #
    # IMPORTANT:
    # This is the Task 1 swept vertical collision logic.
    # Do not replace it with a simple post-movement
    # colliderect() check.
    # -----

    previous_y = self.player.y

    # Move player.
    self.player.y += self.player.vy

    self.player.on_ground = False

    # Calculate player's bottom position before and after
    # movement.
    previous_bottom = (
        previous_y
        + self.player.height
    )

    current_bottom = (
        self.player.y
        + self.player.height
    )

    # Check whether the player's path crossed the top of
    # a platform during this frame.
    for platform in self.platforms:

        platform_rect = platform.rect()

        horizontal_overlap = (
            self.player.x
            < platform_rect.right
            and
            self.player.x
            + self.player.width
            > platform_rect.left
        )

```

```

        crossed_platform = (
            previous_bottom
            <= platform_rect.top
            and
            current_bottom
            >= platform_rect.top
        )

    if (
        self.player.vy >= 0
        and horizontal_overlap
        and crossed_platform
    ):

        self.player.y = (
            platform.y
            - self.player.height
        )

        self.player.vy = 0

        self.player.on_ground = True

        break

# -----
# TASK 2 + TASK 4: Hazard / Death
# -----

    if any(
        self.player.rect().colliderect(
            hazard.rect()
        )
        for hazard in self.hazards
    ):

        self.game_over = True

        # TASK 4:
        # Play death sound exactly when Game Over is triggered.
        self.death_sound.play()

        return

# -----
# TASK 2 + TASK 4: Falling off the screen
# -----

    if self.player.y > self.height:

        self.game_over = True

```

```

    # TASK 4:
    # Play death sound exactly when Game Over is triggered.
    self.death_sound.play()

    return

# -----
# Existing goal/scoring behavior + TASK 4
# -----

if self.player.x >= self.goal_x:

    self.score += 1

    # TASK 4:
    # Play goal sound only when the score actually increases.
    self.goal_sound.play()

    # Existing behavior: reset player after reaching goal.
    self.player.x = self.start_x
    self.player.y = self.start_y

    self.player.vy = 0

# =====
# RENDER
# =====

def render(self, screen):

    # -----
    # Existing world rendering
    # -----

    for platform in self.platforms:

        pygame.draw.rect(
            screen,
            BROWN,
            platform.rect()
        )

    for hazard in self.hazards:

        pygame.draw.rect(
            screen,
            RED,
            hazard.rect()
        )

    goal_rect = pygame.Rect(

```

```

        self.goal_x,
        0,
        6,
        self.height
    )

    pygame.draw.rect(
        screen,
        GREEN,
        goal_rect
    )

    pygame.draw.rect(
        screen,
        WHITE,
        self.player.rect()
    )

    # Score.
    score_text = self.font.render(
        f"Score: {self.score}",
        True,
        WHITE
    )

    screen.blit(
        score_text,
        (10, 10)
    )

    # -----
    # TASK 2 + TASK 3: Game Over / RePlay screen
    # -----

    if self.game_over:

        # Dark transparent overlay.
        overlay = pygame.Surface(
            (
                self.width,
                self.height
            )
        )

        overlay.set_alpha(180)

        overlay.fill(
            (0, 0, 0)
        )

        screen.blit(
            overlay,

```

```

        (0, 0)
    )

    # Game Over title.
    game_over_text = (
        self.game_over_font.render(
            "GAME OVER",
            True,
            WHITE
        )
    )

    game_over_rect = (
        game_over_text.get_rect(
            center=(
                self.width // 2,
                75
            )
        )
    )

    screen.blit(
        game_over_text,
        game_over_rect
    )

    # Final score.
    final_score_text = (
        self.game_over_score_font.render(
            f"Final Score: {self.score}",
            True,
            WHITE
        )
    )

    final_score_rect = (
        final_score_text.get_rect(
            center=(
                self.width // 2,
                140
            )
        )
    )

    screen.blit(
        final_score_text,
        final_score_rect
    )

    # -----
    # Task 3 replay options
    # -----

```

```
easy_text = (  
    self.game_over_option_font.render(  
        "E - Easy",  
        True,  
        WHITE  
    )  
)  
  
medium_text = (  
    self.game_over_option_font.render(  
        "M - Medium",  
        True,  
        WHITE  
    )  
)  
  
hard_text = (  
    self.game_over_option_font.render(  
        "H - Hard",  
        True,  
        WHITE  
    )  
)  
  
exit_text = (  
    self.game_over_option_font.render(  
        "X - Exit",  
        True,  
        WHITE  
    )  
)  
  
screen.blit(  
    easy_text,  
    easy_text.get_rect(  
        center=(  
            self.width // 2,  
            220  
        )  
    )  
)  
  
screen.blit(  
    medium_text,  
    medium_text.get_rect(  
        center=(  
            self.width // 2,  
            270  
        )  
    )  
)  
)
```

```
screen.blit(
    hard_text,
    hard_text.get_rect(
        center=(
            self.width // 2,
            320
        )
    )
)

screen.blit(
    exit_text,
    exit_text.get_rect(
        center=(
            self.width // 2,
            370
        )
    )
)

# Instruction.
instruction_text = (
    self.game_over_instruction_font.render(
        "Choose a difficulty to play again, or exit.",
        True,
        WHITE
    )
)

instruction_rect = (
    instruction_text.get_rect(
        center=(
            self.width // 2,
            435
        )
    )
)

screen.blit(
    instruction_text,
    instruction_rect
)
```

Important: no sound files need to be created

Your project remains:

```
project/
|
└─ main.py
```



```
|  
└─ game/  
    ├── game_engine.py ← MODIFY THIS  
    ├── player.py  
    ├── platform.py  
    └─ hazard.py
```

The three sounds are generated when the game starts:

```
jump_sound  
goal_sound  
death_sound
```



They exist in memory as Pygame `Sound` objects.

Exact testing steps

1. Start the game

From the project root:

```
</> Bash
```



You should see the game normally.

There should be no change to the gameplay screen.

2. Test jump sound

Start the game.

Make sure the player is standing on a platform.

Press:

```
SPACE
```



or:

```
W
```



or:

↑



Expected

You should hear a **short high-pitched sound** when the player jumps.

While the player is already in the air, repeatedly pressing Space should **not continuously trigger jump sounds**.

This is because the code checks:

```
</> Python
```



▶ Run

before playing the sound.

3. Test goal sound

Move right using:

D



or:

→



Reach the green goal line on the right.

Expected

The score changes:

Score: 0



to:

Score: 1



and you should hear the **goal sound**.

The player should then reset to the starting position as before.

The goal sound should happen once for that score increase.

4. Test Game Over through the hazard

Start/replay the game.

Move toward the red hazard.

Touch it.

Expected

You should hear the **low-pitched death sound**.

Then the existing Task 2/Task 3 screen appears:

GAME OVER



Final Score: X

E - Easy

M - Medium

H - Hard

X - Exit

Choose a difficulty to play again, or exit.

The final score should still be correct.

5. Test replay + all difficulties

After Game Over:

Easy

Press:

E



Expected:

- Game restarts.
- Score becomes 0 .
- Player returns to starting position.
- Gravity becomes 0.45 .
- Jump strength becomes -11 .
- Jump sound works.

- Goal sound works.
- Death sound works.

Then deliberately die again.

Medium

At the next Game Over screen press:

M



Expected:

- Game restarts.
 - Score resets to 0.
 - Medium physics are restored.
 - Sounds still work.
-

Hard

At the next Game Over screen press:

H



Expected:

- Game restarts.
- Score resets to 0.
- Hard physics are applied.
- Sounds still work.

The numeric alternatives also remain:

1 = Easy
2 = Medium
3 = Hard
4 = Exit



6. Test falling off the screen

Start/replay the game.

Move toward a gap between platforms.

Do not land on another platform.

Allow the player to fall below the screen.

Expected

The player eventually reaches:

</> Python



 Run

Then:

```

death sound
  ↓
GAME OVER
  ↓
Final Score
  ↓
Easy / Medium / Hard / Exit

```



The death sound should play **once**, not every frame.

7. Verify Tasks 1, 2 and 3

Task 1

Test a long/fast fall toward a platform.

The player should still land using the existing:

```

previous_y
previous_bottom
current_bottom
...

```

</> Python



 Run

logic.

I have **not** replaced this with the old `colliderect()` -after-movement approach.

Task 2

Verify:

Hazard → Game Over



and:

Fall off screen → Game Over



The final score should still be displayed.

Task 3

Verify:

E → Easy
M → Medium
H → Hard
X → Exit



and that a replay resets:

player position
score
velocity
game_over



The sounds are created only once when `GameEngine` is created, so replaying does **not** recreate them every time.

8. Test clean exit

From the Game Over screen press:

X



or:

ESC



The game should close.

You can also close the window using the normal X button.

Your existing `main.py` already ends with:

</> Python



▶ Run

so Pygame's mixer is cleaned up along with the other Pygame modules.

Task 4 changes summary

Task 1

✓ Swept collision detection preserved

Task 2

✓ Graphical Game Over preserved

✓ Final score preserved

✓ Death conditions preserved

Task 3

✓ Easy preserved

✓ Medium preserved

✓ Hard preserved

✓ Replay preserved

✓ Exit preserved

Task 4

✓ Jump sound added

✓ Goal sound added

✓ Death sound added

✓ Sounds generated programmatically

✓ No external packages

✓ No external sound assets

✓ No repeated death sound

✓ No repeated jump sound while airborne

✓ Sounds continue working after replay

Only game/game_engine.py needs to be changed.



ChatGPT can make mistakes. Check important info.